

Neural Pods: versioniertes internes Wissen für reale Sprachmodelle

Kurzfassung. Neural Pods verbinden kanonische Wissensidentitäten, Provenienz, semantische Suche und generation-bound LoRA-Adapter. Ein Pod ist damit kein Textchunk, sondern ein adressierbares Wissensartefakt mit Herkunft, Version, neuralem Payload und Lifecycle-Regeln.

1. Problem

Klassisches RAG behandelt jede Anfrage als neue Textzusammenstellung. Das macht Paraphrasen, wiederholte Nutzung, Updates und Löschung teuer und lässt veraltete Treffer schwer kontrollierbar. Ein internes Wissenselement braucht stattdessen eine stabile Identität und einen überprüfbaren Lebenszyklus.

2. Architektur

Quelle

| OriginKey



Kanonisches Wissen (KnowledgeKey, GenerationKey)

|— Provenienz-DAG

|— Embedding-/BM25-Adresse

|— Dragonfly-Typ-/Alias-Router

|— generation-bound Model-Pods: LoRA, Distilled, MoE, Quantized und Base

|



Qwen3B Reader

|

Lifecycle Barrier

Die vier Identitäten haben getrennte Aufgaben:

| Schlüssel | Bedeutung |

|---|---|

| OriginKey | ursprüngliche Quelle oder Quelle-Version |

| KnowledgeKey | kanonische semantische Identität |

| GenerationKey | exakte gültige Version |

| ArtifactKey | abgeleiteter Vektor, Adapter, Cache oder Zustand |

Ein Alias ist damit nur ein weiterer Zugang zu demselben KnowledgeKey. Eine

Revocation propagiert deterministisch über die abhängigen ArtifactKeys.

3. Implementierung im Prototyp

Der lokale Backend-Pfad implementiert ohne externe turbopuffer-API:

typed namespaces und Metadaten

lokale Native Embeddings mit dem geprüften 384-dim Encoder

ANN-/kNN-ähnliche Vektorsuche, BM25, Filter und RRF-Multi-Queries

Branching, WAL, Partial Updates und Re-Embedding

Dragonfly-Routing nach Pod-Typ, Domain, Rolle und Intent

generation-bound Model-Pods und Revocation-Barriere

Die Model-Pod-Varianten sind lora, distilled, moe, quantized und base. Distilled-Pods tragen eine Teacher-Identitaet, MoE-Pods ein Experten-Manifest; alle Varianten bleiben an KnowledgeKey und GenerationKey gebunden.

4. Reale Messungen

| Messung | Ergebnis |

|---|---:|

| Qwen3B-LoRA auf RTX 3090 | 23,9 Token/s |

| Pod-Kontext vs. RAG-Kontext | 61 vs. 303 Eingabetoken |

| Qwen-Latenz, Batch 4 | 190,9 ms vs. 412,5 ms |

| Lokale Suche, 2.000 Rows | 9,48 ms p50; 101,7 QPS parallel |

| Lifecycle-Manifest | 0,052 ms p50 |

| R211b inkrementelles Update | 434× gegenüber Vollkompilierung |

| Native-Embedding-Recall@1 | 1,0 auf dem lokalen Fixture |

| Boundary-Reader | 86/96 roh, 100/100 mit typisiertem Guard |

Die Qwen-Zeitmessung verwendet denselben Qwen3B-LoRA-Reader und dieselbe GPU.

Der Pod-Pfad enthält kompakte typisierte Aktivierungsmetadaten; der RAG-Pfad

enthält längere wiederholte Evidenz. Die Messung ist daher ein

Repräsentations-/Prefill-Vergleich, kein allgemeiner Beweis für bessere

Antwortqualität.

5. Lifecycle-Eigenschaft

K:g7 → Pod:A91 → Cache:C144

|

└ revoke

↓

K:g8 → Pod:A92

Ein alter Pod kann nach einer Revocation nicht mehr autorisieren. Der Registry-

Snapshot, die Adapter-Manifest-Prüfung und die Suchfilter müssen dieselbe

Generation sehen. Genau diese Invarianten sind im Projekt-Gate automatisiert.

6. Einordnung und nächste Prüfung

Der Prototyp zeigt eine funktionierende Integration auf einem echten Qwen3B-

Modell und reproduzierbare Lifecycle-Mechanismen. Noch offen sind ein fairer

Qualitätsbenchmark gegen optimiertes verteiltes RAG, größere natürliche

Multi-Hop-Datensätze, vLLM/ROCM-Serving und echte 100B-ANN-Skalierung.

Die zentrale Hypothese für die nächste Studie lautet:

Ein generation-bound Neural Pod kann wiederholt genutztes internes Wissen

mit weniger Kontext und kontrollierbarerem Lebenszyklus als eine reine

Retrieval-Pipeline bereitstellen.

7. Rekursive Exploration als Pod-Schicht

Historische Discovery-Trees werden offline replaybar; Broad-to-Deep-Erkundung vertieft nur unsichere Äste. Ausschließlich verifizierte Action-Condition-Outcome-Beziehungen werden als Experience-Pods konsolidiert. Der Demo-Lauf replayt zwei Fälle mit 2/2 korrekten Ergebnissen und bindet sie über das Registry-DAG.

Reproduzierbarkeit

Gate: `runs/project-gate-001.json` (20/20)

Stack-Inspektion: `research/STACK-INSPECTION-20260916.md`

Pod/RAG-Messung: `runs/multifact-internal-lora-004/pod-vs-rag-throughput.json`

End-to-End-Demo: `runs/assembled-stack-demo-001.json`